



Calibrating LLMs

Calibrating LLMs (Confidence Calibration)

1) What it is & why it matters

Calibration means aligning a model's stated confidence with its **true likelihood of being correct**.

- A well-calibrated system says "90%" only when it's right $\approx 90\%$ of the time.
 - Benefits: **trust**, **risk-aware UX**, **routing/triage** (human-in-the-loop), and **policy/compliance** decisions.
-

2) Where "confidence" comes from in LLMs

- **Classification/logit-based**: softmax over labels or token logprobs for choices (e.g., "Yes/No").
- **Sequence likelihood**: normalized log-probability of a candidate string.
- **Elicited confidence**: model self-reports a number (needs calibration).
- **Proxy signals**: agreement in **self-consistency** (vote fraction), retrieval score, verifier score, etc.

| Treat raw numbers as uncalibrated until validated.

3) Metrics you'll use

- **ECE (Expected Calibration Error)**: average gap between accuracy and confidence across bins.
- **MCE (Max Calibration Error)**: worst bin gap.
- **Brier Score**: mean squared error of probabilities vs. outcomes.

- **NLL / Cross-Entropy**: penalizes overconfidence.
 - **Reliability diagram**: plot confidence vs. empirical accuracy.
-

4) Post-hoc calibration methods (no model retraining)

- **Temperature scaling** (multi-class): learn a scalar T on a dev set; apply to logits $\rightarrow$ softer probabilities.
- **Platt scaling** (binary logistic regression on scores).
- **Isotonic regression** (monotonic, non-parametric mapping).
- **Vector/Matrix/Dirichlet scaling** (richer but heavier).
- **Histogram binning** (simple baseline).
- **Conformal prediction** (coverage-controlled *sets* of answers rather than single probabilities; gives $1-\alpha$ guarantee under exchangeability).

Which to choose?

Start with **temperature scaling** (fast, strong baseline). If strong class imbalance or nonlinearity $\rightarrow$ **isotonic** or **Dirichlet**. Use **conformal** when you need guaranteed coverage and can return *abstain/sets*.

5) Selective prediction & abstention

Calibrated confidence lets you **abstain** when unsure.

- Choose a **threshold τ** on a dev set to hit target **coverage** (fraction auto-answered) at desired **risk** (error among auto-answered).
 - UI pattern: "Low confidence—escalating to a human," or return **TOP-k set** (conformal) instead of a single guess.
-

6) Practical recipes

6.1 Classification (labels or "Yes/No")

1. Collect dev set with ground truth.
2. For each item, get **logits/logprobs** for labels.
3. Fit **temperature scaling** (optimize NLL).

4. Evaluate with **ECE/Brier**; draw reliability curve.
5. Pick τ for accept/abstain; log operating point (coverage, risk).

6.2 Extraction / structured outputs

- Train a **verifier** (LLM or small classifier) that checks schema validity + evidence.
- Convert verifier score to probability and **isotonic-calibrate** it.
- Accept only if $p \geq \tau$; otherwise **repair** or escalate.

6.3 Open-ended generation (QA with citations)

- Derive confidence from a **judge**: groundedness (citation match), contradiction checks, and format adherence.
- Aggregate signals (e.g., weighted sum) → **calibrate** with Platt/Isotonic.
- Optionally add **self-consistency**: vote fraction across k samples → calibrate that fraction to accuracy.

7) Prompt & output design (copy-paste)

System (confidence + structure)

```
Return ONLY valid JSON:
{
  "answer": "<text or label>",
  "confidence": 0.00-1.00,    // your best estimate before c
alibration
  "evidence": ["<short quotes or citation IDs>"],
  "validation": { "format_ok": true|false, "grounded": true
|false }
}
If evidence is insufficient, set confidence ≤ 0.3 or output
"abstain".
```

Controller (thresholding)

```
if json.validation.format_ok && json.validation.grounded
  p = calibrate(json.confidence or verifier_score)  # le
```

```

    arned mapping
    if p >=  $\tau$ : return json.answer
    else: route_to_human()
else: try_repair_or_abstain()

```

8) Implementation blueprint

1. **Data**: build a labeled dev + hold-out set that matches deployment distribution.
2. **Score**: collect raw confidences (logits, judge scores, vote fractions, self-reports).
3. **Fit**: learn a calibration map (Temp/Platt/Isotonic).
4. **Evaluate**: ECE, Brier, reliability plots; test **drift** sensitivity.
5. **Decide**: set τ for coverage/quality; define abstain UX.
6. **Monitor**: online ECE proxy, error among accepted, escalation rate, and drift; re-calibrate periodically.

9) Common pitfalls & fixes

Pitfall	Fix
Calibrating on tiny/misaligned data	Use representative, temporally recent dev sets; stratify by domain
Treating self-reported numbers as calibrated	Always fit a mapping; compare to ground truth
One τ for all tasks	Use task-/route-specific thresholds; segment by domain
Ignoring drift	Schedule recalibration; track coverage/accuracy over time
Overconfident LLM judges	Penalize lack of evidence; require citations; ensemble judges

10) Monitoring checklist

- ☐ ECE / Brier on rolling windows

- ☐ Coverage vs. risk (accepted vs. abstained)
 - ☐ Calibration per **slice** (domain, language, length)
 - ☐ Judge disagreement rate / self-consistency gap
 - ☐ Retraining/recalibration cadence documented
-

Key Takeaway

Calibration doesn't make a model *smarter*—it makes it **honest** about uncertainty. Fit a simple post-hoc mapping on a representative dev set, set abstention thresholds, and monitor drift so users and downstream systems can **trust** the probabilities you show.